Interpreting Transformer Representations on Rubik's Cube Tasks

Cole McGuire

For my final project, I want to use Rubik's cube as a small, controlled setting for interpretability. Instead of just asking whether a model can solve the cube, I want to look at what it is representing internally while making decisions. I think the cube is a nice test case because the state is fully known, legal moves are easy to define, and progress toward the goal is much clearer than in natural language. There has already been work on solving Rubik's cube with reinforcement learning, self-supervision, and transformers, but most of that work focuses on performance rather than interpretability [1–3].

My plan is to generate a dataset of cube states and move sequences, probably using the $2\times 2\times 2$ cube so the project stays manageable. Then I will train or fine-tune a small transformer on a task like next-move prediction or predicting whether a move helps. After training, I will use linear probes to see whether the model's hidden states contain information about things like scramble depth, whether a simple subgoal is complete, or whether a candidate move improves the position. If I have enough time, I may also try a tuned-lens-style analysis to see how the model's predictions change across layers [4, 5].

My main hypothesis is that the model will learn useful internal representations of cube progress, and that these will become easier to decode in later layers. I expect simple concepts, like scramble depth or whether part of the cube is solved, to be easier to recover than the full cube state. To test this, I will train probes on activations from each layer and compare their accuracy across several labels. If the results are strong enough, I may also try a small intervention experiment to see whether changing activations along a probe direction changes the model's move preferences. I would change my mind if the probes stay near baseline or if later layers are not any more informative than earlier ones, since that would suggest the model is not building the kind of structured internal representation I expect [4, 6].

The biggest challenge will probably be scope. Training a good cube model from scratch could take longer than I expect, so I may need to simplify the task or reduce the number of experiments. Another risk is over-interpreting probe results, since a probe can show correlation without showing that the model actually uses that feature causally. If the full version is too ambitious, I can narrow the project to a simpler question about whether scramble depth and a few local subgoals can be decoded reliably from a small cube model [6].

As backup options, I am also considering two related directions. One would be to use Rubik's cube as an evaluation benchmark for language-model reasoning by testing how well a model can predict legal moves, estimate progress, or explain why a move is good or bad. The

other would be a more ambitious sparse autoencoder project, where I would train an SAE on the transformer's hidden states and see whether sparse features line up with interpretable cube concepts. Right now, though, the probing-based project seems like the best balance between being interesting, doable, and relevant to interpretability.

References

[1] F. Agostinelli, S. McAleer, A. Shmakov, and P. Baldi, "Solving the rubik's cube with deep reinforcement learning and search," *Nature machine intelligence*, vol. 1, no. 8, pp. 356–363, 2019.

[2] K. Takano, "Self-supervision is all you need for solving rubik's cube," *arXiv.org*, 2023.

[3] M. E. Chasmai, "Cubetr: Learning to solve the rubiks cube using transformers," *arXiv.org*, 2023.

[4] G. Alain and Y. Bengio, "Understanding intermediate layers using linear classifier probes," *arXiv.org*, 2016.

[5] N. Belrose, I. Ostrovsky, L. McKinney, Z. Furman, L. Smith, D. Halawi, S. Biderman, and J. Steinhardt, "Eliciting latent predictions from transformers with the tuned lens," *arXiv.org*, 2025.

[6] Y. Belinkov, "Probing classifiers: Promises, shortcomings, and advances," *Computational linguistics - Association for Computational Linguistics*, vol. 48, no. 1, pp. 207–219, 2022.